

Homework - Categorization and Model Comparison Part A (70/105 points)

PSY/COS 360, Computational Models of Cognition, Fall 2026.

Credit for first version and for the categorization data: Prof. Todd Gureckis at New York University.

Parts of your code are checked by an automated grader. For the grader to work, please keep the following in mind as you complete the assignment:

- **Do not rename this notebook file or any of the starter functions we provide.** The grader looks for them by name, so a renamed file or function will fail even if the code is correct.
- **Make sure the notebook runs from top to bottom without errors.** Before submitting, choose `Restart`, `Clear All Outputs`, and then `Run All` from the notebook menu. Confirm that every cell executes in order and produces the expected output.
- **Do NOT clear the output when submitting.** Submit the notebook with the output of each cell. The grader may use these values.

```
In [24]: import os
import numpy as np
import seaborn as sns
import pandas as pd
import math
from scipy.optimize import fmin
```

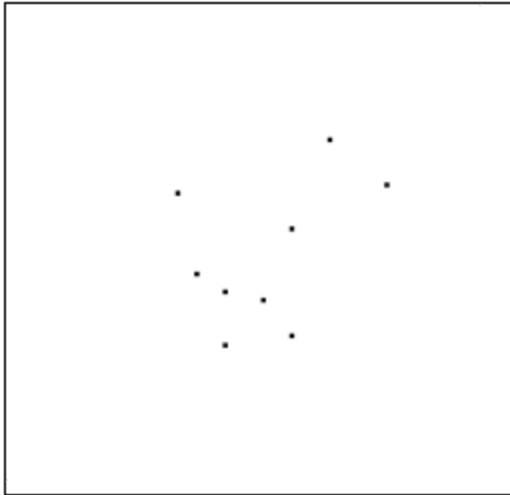
Background and Theory

In this homework we explore the cognitive mechanisms that support unsupervised pattern categorization in humans. In addition, we use this as an example of testing and comparing between models.

A simple (classic) unsupervised categorization experiment

Posner and Keele (1968) report a now classic categorization experiment with humans. In the task participants viewed visual stimuli that are clouds of points (known as dot

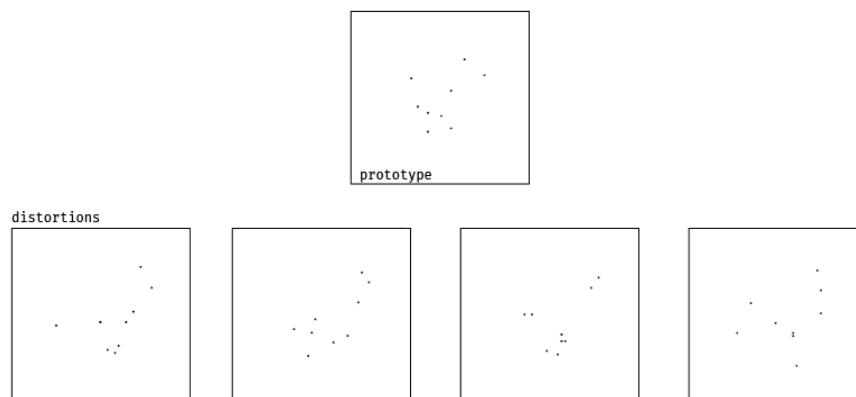
patterns) similar to a scatter plot of data on a graph. An example of the stimuli is shown here:



Experiment Design

Their experiment went like this, with some simplifications from their original procedure. The experiment was divided into a training and test phase. During the training phase, for each subject a single random dot pattern was generated and considered to be the underlying "prototype" structure. A prototype is like a common template or reference pattern. The key is that participants never get to see the "prototype" pattern directly during training. Instead they see what are known as "distortions" of the prototype. A distortion of a pattern is made by adding random spatial noise to each point in a pattern to kind of "wobble" the points away from their original position.

For example, here is a random prototype (top) and a bunch of random distortions of the prototype made by adding or subtracting small random values from the `<x,y>` value of each point in the pattern.



There are distortions that added more or less random noise. For example, "high" distortions add a lot of randomness to the underlying template pattern whereas "low" distortions add only a little bit of noise.

Training Phase

In the training phase of the experiment subjects view 10 training examples one at a time which are "high" distortions of a randomly generated prototype. The instructions are that subjects should look at these patterns, and that they come from a single category similar to if you viewed a series of pictures of dogs they would all come from the category `dog`. Subjects were trying to figure out the pattern that related the different images to one another. Try it for yourself by looking at each of the "distortions" patterns above one by one and trying to detect the common structure.

Test Phase

During the test phase, participants view a series of dot patterns one at a time and have to judge: **Does the given pattern come from the same general category or family you studied earlier or is it a new pattern that is different?** This is an unsupervised categorization task because the subject has to abstract what the common structure is from the given patterns and then use that information to make classification decisions about new patterns.

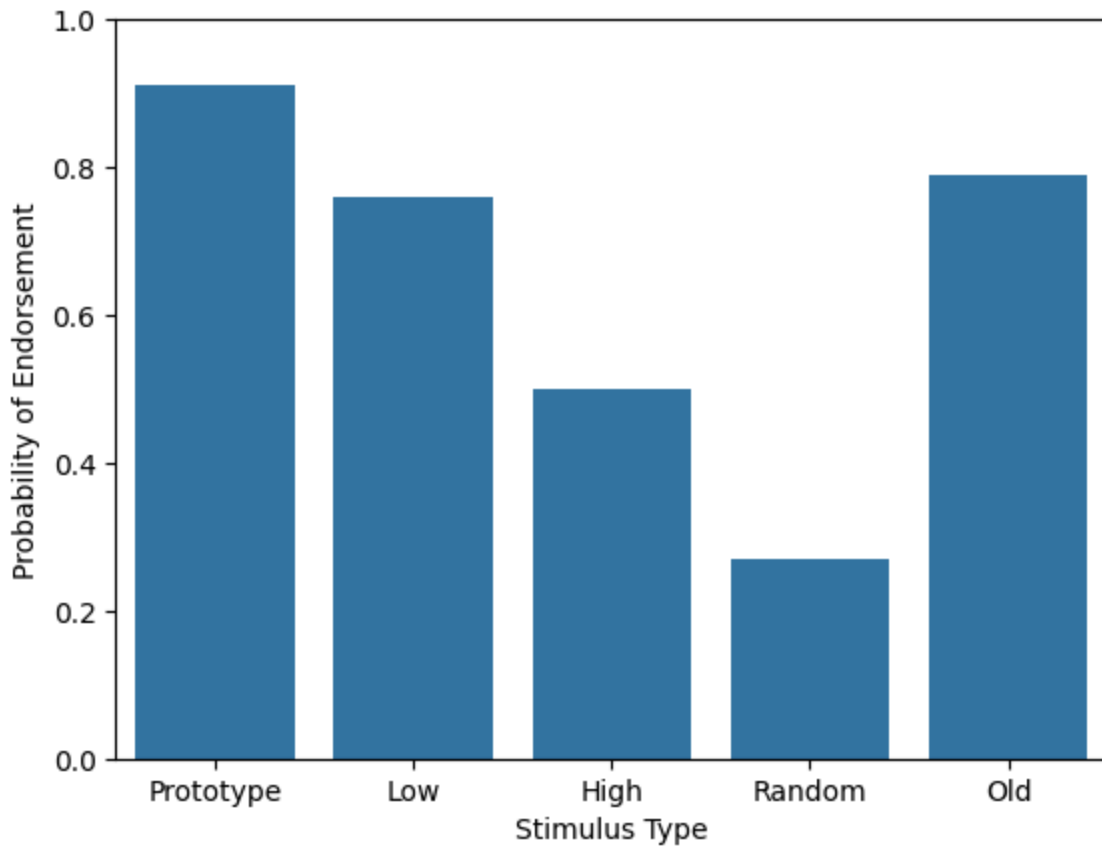
Unknown to participants the set of test items varied in a specific way with respect to the training patterns. In particular, there were five particular types of patterns presented during test.

- The first type were "old" patterns which were identical to those presented during the training phase.
- The second type were "random" patterns which were from a completely new randomly generated prototype (thus had nothing to do with the items presented during training).
- The third type were new "high distortions" of the underlying prototype that was used to create the study set. These are thus similar to the "old" items but do not match exactly.
- The fourth type were "low distortions" of the underlying prototype that was used to create the study set. These are more similar to the prototype pattern than the "high" distortions are.
- Finally the actual prototype used to generate the items during training was presented. This pattern is interesting because the prototype pattern was never seen exactly during training. However, people saw many high distortions of this item during training and given the instructions to detect what the common structure of the training patterns is, they may have learned something about this latent or hidden pattern.

Typical Results:

This graph shows example results that are typical for an experiment like this:

```
In [25]: df = pd.DataFrame({"Stimulus Type": ['Prototype', 'Low', 'High', 'Random', 'Old']  
ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", data=df)  
ax.set_ylim(0, 1.0);
```



The height of the bars indicates the probability of endorsing a pattern as a member of the category during the test phase (high values mean that at test a subject is more likely to agree "yes this pattern fits with the one I learned"). Notice that the "old" items (the exact patterns studied during learning) are endorsed at a relatively high rate. In contrast the "random" patterns (those coming from a completely different underlying pattern) are endorsed at a very low rate. The low and high distortions are endorsed at intermediate rates (with the low distortions preferred). Interestingly, the prototype pattern is endorsed most strongly even though it was never presented during the study period! It is like during learning people figured out the underlying pattern that generated the stimuli!

Categorization: Memory for examples or abstractions?

The dot pattern experiments are interesting and have been replicated in various ways perhaps hundreds of times. How do people make these judgments? What information do

people store during the study phase that would predict their performance in the test phase? What rule do they use for combining information from memory in order to make these kinds of judgments? We are going to build up a model of categorization in a few simple steps.

Step 1: How are dot patterns represented in the mind?

Our first question concerns how people represent the dot patterns and the distances between different dot patterns. Based on work from Smith and Minda (2002), let's assume the relationship between the physical distance of two dot patterns i and j (average distance between pairs on the screen) and the psychological distance between them d_{ij} (how these distances are perceived) is as follows:

$$d_{ij} = \log\left(1 + \frac{\sum_m \sqrt{(i_{mx} - j_{mx})^2 + (i_{my} - j_{my})^2}}{9}\right)$$

where i_{mx} is the x position of the m -th dot in pattern i and j_{mx} is the x position for pattern j (likewise for the y position, i_{my} and j_{my}). There are 9 dots total, so we divide by 9 to get the average. Because it can sometimes be ambiguous which dot aligns with which one in a pattern we choose the dots which are closest in the two patterns to compute this score.

Step 2: What is stored in memory?

The next consideration is what people actually store in memory during the training phase of the experiment. There are of course many alternatives. People could store an "average" of the points seen so far, or they could store each individual pattern that they have seen, or they could store nothing and try to figure it out at test, or they could store some verbal description of what the shapes "look like", or the shape of the outer edge of the dot-cloud (the "convex hull"), etc...

There are, however, two leading theories which have attracted considerable debate in the cognitive science literature: the prototype and exemplar theory.

Exemplar models

Exemplar models are a general class of psychological models related to nearest neighbor algorithms. The most important feature of these models is the idea that people have what appears to be a nearly infinite memory for the past and as a result you can store all past experiences or examples in memory. This seems crazy as we are forgetting things all the time but actually psychology is unclear about if we actually forget things or if we

simply lose the ability to retrieve a memory (i.e., more like losing the pointer to the memory rather than decay).

In machine learning, nearest neighbor classifiers use a distance function (like the one described above) to retrieve from memory the nearest labeled example and to predict the category membership based on the label for this item. This nearest neighbor algorithm can be relaxed slightly to consider k -nearest neighbors. According to this algorithm you find the k neighbor examples (with $k > 1$) to the current pattern and respond based on what the majority of these examples say.

Now we can go a bit further and say that you compute the distance to all past examples but *weight* their vote according to how close they are. So instead of picking the label of the closest or k -closest examples we compare the current pattern using a global match to all examples in the memory and weight their predictions based on similarity. This is an exemplar model... Pretty neat!

Ok, but how does this help us in the case of **unsupervised** categorization such as in the dot pattern case? Here what we will assume is that we compute the similarity (written as η_{ij} in the lecture slides) of the to-be-categorized item i (the test item) to all the examples j stored in memory and compare it to some criterion value. If the sum of the similarities to all the examples is below this criterion then we assume the pattern is new and doesn't match what we learned. If it is above the criterion we judge the item is a good example of the category.

We'll assume similarity takes the following form,

$$\eta_{ij} = e^{-c \cdot d_{ij}}.$$

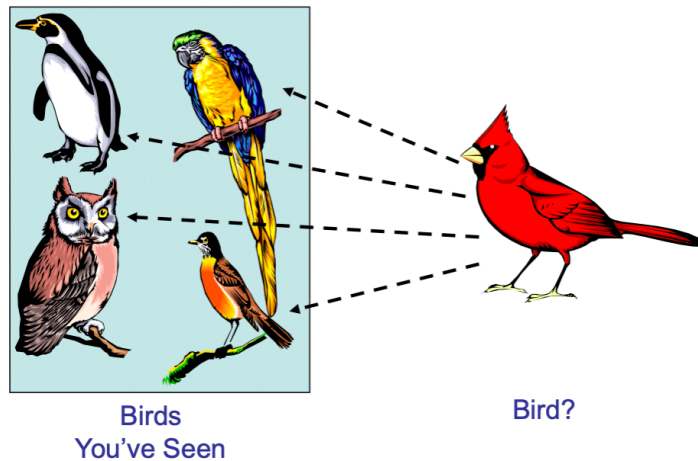
Again, d_{ij} is the psychological distance between pattern i and pattern j which is an example stored in memory during the study phase. This form has deeper relation to research on categorization that we'll discuss in class with regards to Shepard's universal law of generalization. c is a free parameter that controls that weighting function and is often fitted to data.

In the (unsupervised) exemplar model we will consider that the probability of endorsing an item is going to be determined by the following equation:

$$P(A|i) = \frac{\sum_j e^{-c \cdot d_{ij}}}{\sum_j e^{-c \cdot d_{ij}} + k}$$

where $P(A|i)$ is the probability of endorsing pattern i as a member of the category seen during study. k is the criterion against which the summed similarity ($e^{-c \cdot d_{ij}}$) is being compared. If k is zero then you endorse the item as a member of the category all the time irrespective of the similarity and if k gets large you become less and less likely to

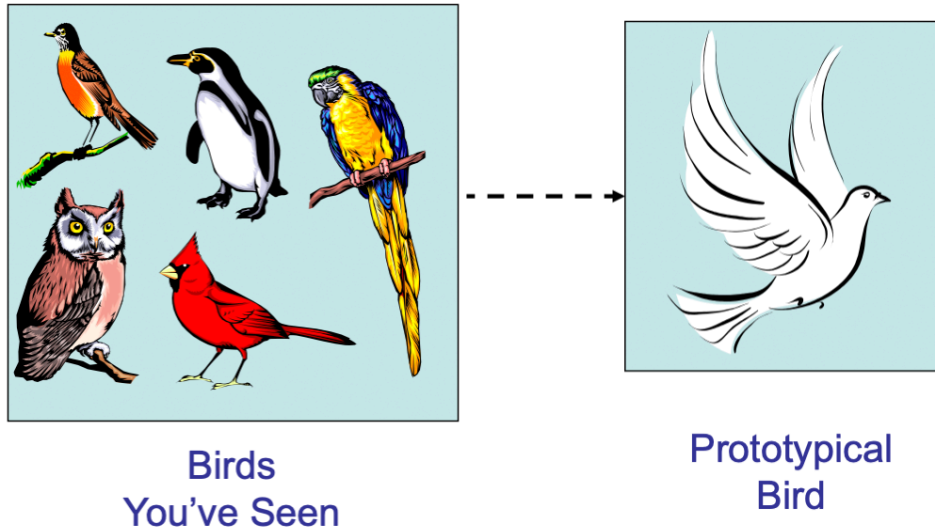
endorse the item (i.e., more likely to say no). You can think of it as the particular weighted nearest neighbor algorithm we think the mind uses. Note that there are no β_A and β_B bias parameters as in the lecture notes, which is used for making a two-way classification between A and B, which is not the scenario here.



Together, this model implements the idea that very close matches ($d_{ij} = 0$) are especially strong and things that are less similar count less. You can think of it as the particular weighted nearest neighbor algorithm we think the mind uses.

Prototype models

A prototype model is different from an exemplar model because it assumes that instead of storing each of the training patterns in memory exactly, instead people store a single summary representation. For example, people might store a mentally computed "average" pattern. When you think about how you would perform the task you might think that you kind of compare the training patterns to one another and then compute some summary.

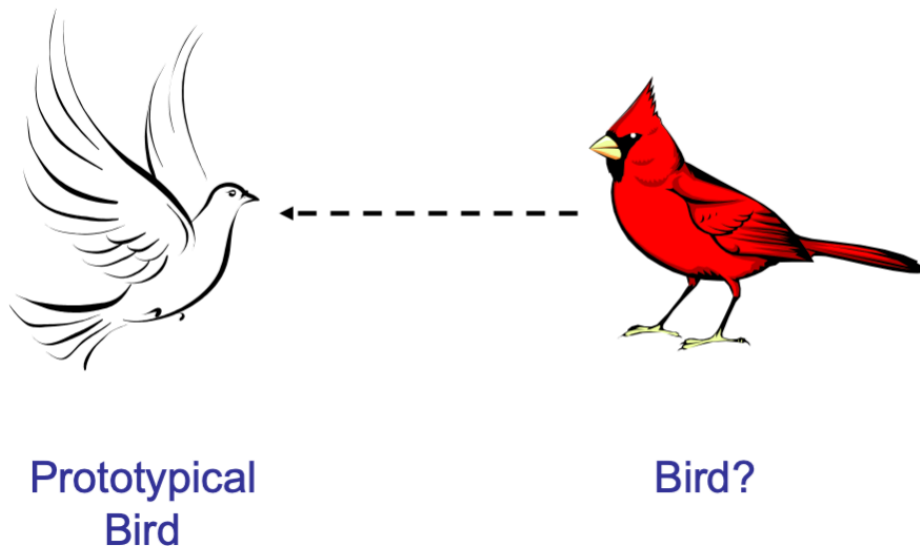


In the case of the dot pattern stimuli one way to do this is to store a special trace in memory called the prototype which is the average of all the patterns seen so far (averaging the $\langle x, y \rangle$ position of each point to find an average dot location).

There are many different ways to operationalize a prototype model. According to the one we use here, the probability of endorsing a test item pattern as a member of the category that was studied during training is:

$$P(A|i) = \frac{e^{-c \cdot d_{ip}}}{e^{-c \cdot d_{ip}} + k}$$

Note that nearly everything about this equation is the same except there is no longer a sum! Instead we simply compute the distance between the test pattern and this special "prototype" pattern (p) which has been averaged from the training examples.



Parameters

k and c are "free parameters" in both the exemplar and prototype model which are assumed to modulate or alter the core psychological processes. These parameters might vary between subjects and as a function of condition. Thus, in order to assess the ability of the model to account for the data we often "fit" these parameters to our data.

Model Comparison

With these ideas in mind, in this homework you are going to compare the exemplar and prototype model to account for some data from an actual dot pattern categorization task collected with human subjects. The goal is that by doing the homework you would develop some useful code that would let you more or less plug in a model that you might come across in your research, fit it to data, and verify that the fits are good, etc...

Reading in some data

The `data/` folder that comes with this homework contains data from 14 human subjects who participated in a dot pattern classification task. The data describing each subject is in a text file (`.dat`) indexed by subject number (e.g., `1.dat` , `2.dat` , etc...).

The organization of these files is as follows:

The first 44 lines of the file contain a description of the stimulus that the subject saw on a given trial including the x, y coordinate of each dot. The first columns of these 44 lines is the number of the pattern (`1-44`). The second column is the type of pattern using the following codes:

- `1` = prototype
- `2` = 10 "high distortions" of the prototype that were used as study patterns during learning
- `3` = 10 new "high distortions" of the prototype presented during test
- `4` = 4 "low distortions" of the prototype that were presented during test
- `5` = 20 random items that come from different prototypes that were presented at test

The next 18 values of each row are the coordinates of the dots (with the x, y coordinates in sequence). So `[x1, y1, x2, y2, x3, y3, ...]` .

The following 40 lines of the file show the sequence of items presented during the study phase. This is not all that important for our purposes, but basically the last column is which pattern was displayed (indexed from the patterns just described). Each of 10 "high distortions" were presented four times each during study in a random order.

Finally the remaining lines of the file report the results of the test phase. The first column is the subject number, the second column is the condition number, the next is the trial number in the experiment, the other columns worth mentioning are the last column (the pattern number from the beginning of the file), the second to last column (the type of stimulus it is according to the codes described above), and the reaction time in milliseconds.

In addition, participants in this experiment were assigned to one of two conditions: a recognition condition and a categorization condition. These conditions differed only in the instructions given to participants at the start of the test phase. In the recognition condition participants were told they would view a series of patterns and they should respond "yes" only if the pattern was **exactly** one they saw in the previous study phase. In the categorization condition, participants were asked to respond "yes" only if the pattern belonged to the same general category or pattern that they observed in the training phase.

The following graph computes the probability of endorsement in the data set as a function of stimulus type and condition (Cat or Rec instructions).

```
In [26]: #####
# getcurve
#####
def getcurve(filename):
    prototypes = []
    low = []
    old = []
    high = []
    random = []
    mydata = readfile(filename)
    cond = mydata[-1][1]
    for line in mydata:
        if line[4] == 2 and len(line) == 9:
            if line[7] == 1:
                prototypes.append(line[5])
            if line[7] == 2:
                old.append(line[5])
            if line[7] == 3:
                high.append(line[5])
            if line[7] == 4:
                low.append(line[5])
            if line[7] == 5:
                random.append(line[5])

    #print([len(prototypes), len(low), len(high), len(random), len(old)])
    # print(prototypes)
    # print(low)
    # print(high)
    # print(random)
    # print(old)
    return [np.average(prototypes), np.average(low), np.average(high), np.av
```

```

def readfile(filename):
    results = []
    fp = open(filename, 'r')
    for line in fp.readlines():
        myline = list(map(int, line.split(' ')[:-1]))
        results.append(myline[:])
    fp.close()
    return results

def get_all_filenames(directoryname):
    files = filter(lambda x: x[-4:] == '.dat' and x[0] !=
                    '.', os.listdir(os.path.join(".", directoryname)))
    fn = map(lambda x: os.path.join(".", directoryname, x), files)
    # process each file and drop last 5 trials
    return list(fn)

def create_df(subjnum, cond, pattern):
    nobs = len(pattern)
    df = pd.DataFrame({"Subject": [subjnum]*nobs, "Condition": [cond]*nobs,
                      'Prototype', 'Low', 'High', 'Random', 'Old'], "Probability")
    return df

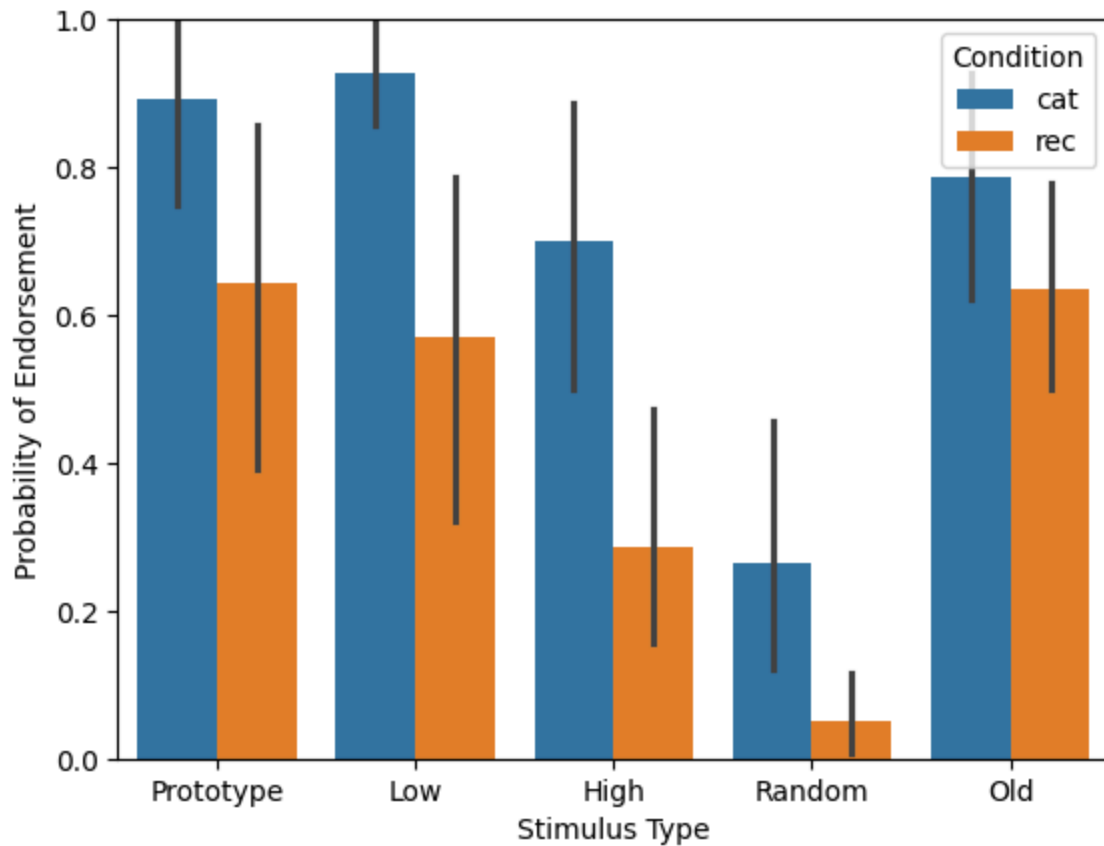
def get_human_results():
    allres = map(getcurve, get_all_filenames('data'))
    cat = []
    rec = []
    for patt in allres:
        if patt[-1] == 0:
            cat.append(create_df(patt[-2], 'cat', patt[:-2]))
        else:
            rec.append(create_df(patt[-2], 'rec', patt[:-2]))
    cat, rec = pd.concat(cat), pd.concat(rec)
    return pd.concat([cat, rec])

```

```

In [27]: ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", hue="Condition")
ax.set_ylim(0, 1.0);

```



Problem 1 (20 points)

Using your own words explain the data pattern you see in the above figure. What is different between the conditions and stimulus type? Why do you suspect these patterns exist? Your answer will need to consider the nature of the experiments, what is manipulated, and even your intuitive psychological theory about what might be going on. Your response should take 3-4 sentences and appear in a cell below. Is any feature of this data surprising to you?

Answer From the graph we can see that endorsement drops as test patterns become less similar to the studied category. Prototype, low, and old items are endorsed most often while high distortions less so, and random items least. Endorsement is lower overall in recognition that in categorization which makes sense because the conditions differ only in their instructions. Recognition asks for an exact match to a studied pattern whereas categorization ask only for the general category. We can see that "random" category is endorsement the least from all since it is made to be vastly different than what was shown.

Predictions for the exemplar model

The following cells set up the exemplar model using the equations described above.

```
In [28]: #####
# unitdist:
# computes the euclidean distance between
# two dots
#####
def unitdist(x, y):
    x1 = np.array(x)
    y1 = np.array(y)
    return math.sqrt(sum(pow(x-y, 2.0)))

#####
# computeresponse
# computes the "activation" of each
# trace in memory
#####
def computeresponse(target, memory, c, k):
    res = []
    for mem in memory:
        res.append(
            math.log(1.0+np.average(list(map(lambda x, y: unitdist(x, y), ta
resp = [math.exp(-c*x) for x in res]
pofr = sum(resp)/(sum(resp)+k)
    return pofr
```

```
In [29]: #####
# exemplar model
# stores all 10 study items in memory
# and computes the probability of endorsement
# for each item type
#####

def exemplarmodel(filename, c, k):
    data = readfile(filename)
    cond = data[-1][1]
    memory = []
    for line in data:
        if len(line) == 20 and line[1] == 2:
            memory.append(np.resize(line[2:], (9, 2)))
    # print(memory)

    # prototype items
    proto = []
    for line in data:
        if len(line) == 20 and line[1] == 1:
            item = np.resize(line[2:], (9, 2))
            pofr = computeresponse(item, memory, c, k)
            proto.append(pofr)
    # print(np.average(proto))

    # old items
    old = []
```

```

for line in data:
    if len(line) == 20 and line[1] == 2:
        item = np.resize(line[2:], (9, 2))
        pofr = computeresponse(item, memory, c, k)
        old.append(pofr)
# print "p of r", old
# print(np.average(old))

# new high items
newhigh = []
for line in data:
    if len(line) == 20 and line[1] == 3:
        item = np.resize(line[2:], (9, 2))
        pofr = computeresponse(item, memory, c, k)
        newhigh.append(pofr)
# print(np.average(newhigh))

# new low items
newlow = []
for line in data:
    if len(line) == 20 and line[1] == 4:
        item = np.resize(line[2:], (9, 2))
        pofr = computeresponse(item, memory, c, k)
        newlow.append(pofr)
# print(np.average(newlow))

# random items
random = []
for line in data:
    if len(line) == 20 and line[1] == 5:
        item = np.resize(line[2:], (9, 2))
        pofr = computeresponse(item, memory, c, k)
        random.append(pofr)
# print(np.average(random))

return [np.average(proto), np.average(newlow), np.average(newhigh), np.a

```

```

In [30]: def get_exemplar_results(c_cat, k_cat, c_rec, k_rec):
allres = {fn: readfile(fn) for fn in get_all_filenames('data')}
cat = []
rec = []
for filename in allres.keys():
    if allres[filename][-1][1] == 0:
        res = exemplarmodel(filename, c_cat, k_cat)
        cat.append(create_df(filename, 'cat', res[:-2]))
    else:
        res = exemplarmodel(filename, c_rec, k_rec)
        rec.append(create_df(filename, 'rec', res[:-2]))
cat, rec = pd.concat(cat), pd.concat(rec)
return pd.concat([cat, rec])

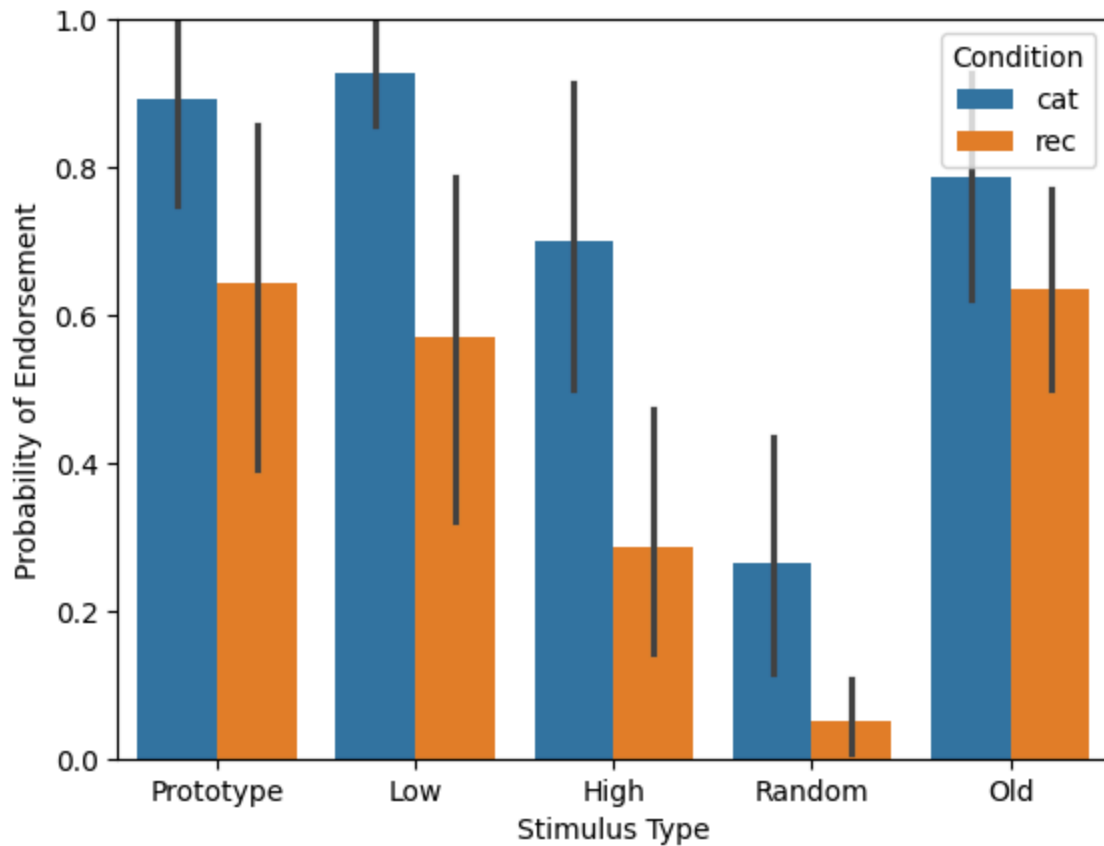
```

First let's replot the human results:

```

In [31]: ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", hue="Cor
ax.set_ylim(0, 1.0);

```



Problem 2 (10 points)

By hand adjust the setting of the model parameters to roughly fit the human data pattern shown above. How close can you get? What parameters did you find (report them) and your assessment of how well they fit. Was it a good fit or are there systematic problems with the fit? In addition, what are the parameter values and do they make sense in light of the equations described above? When the parameters are the same for recognition and categorization instructions why do the bars look a little different?

Answer

Best fit parameters found: $c_{cat} = 2.0$, $k_{cat} = 0.1$, $c_{rec} = 2.0$, $k_{rec} = 0.5$

Using the parameters above, I was able to get decently close to that of the presented graph. We can see the clear difference for randomness, and the bars for prototype are also pretty equivalent. However, something worth noting is that "old" was a little hard to adjust and that is because the model over-predicts "old" items. This is because since an exemplar model every training example is stored and used to compare new items, an old item will have a stored copy at $d = 0$ adding a full $e^0 = 1$ to the summed similarity. This

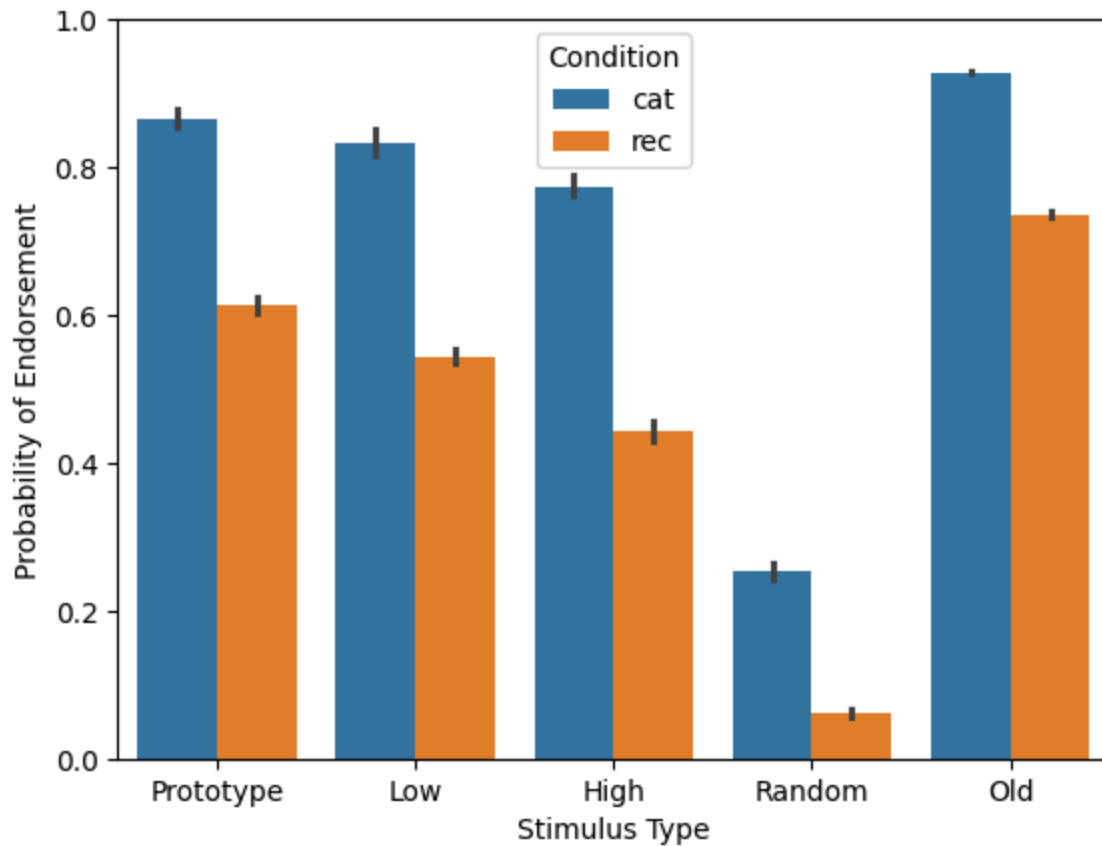
is why its bar is above prototype. The parameters do make sense since from the equation

$$P(A|i) = \frac{\sum_j e^{-c \cdot d_{ij}}}{\sum_j e^{-c \cdot d_{ij}} + k}$$

, we know that c roughly sets the spread of the bars. In this case setting $c = 2$ for both recognition and categorization lets similar patterns contribute some effect to the spread but keeps far away patterns near zero. We know that k is the criterion against which the summed similarity ($e^{-c \cdot d_{ij}}$) is being compared, so a small $k = 0.1$ gives high endorsement for categorization while a $k = 0.5$ for recognition is much stricter since recognition is seeing exactly as trained, hence bars are dropped more. Even when the params are the same for recognition and categorization instructions, they still differ since the two conditions contain different subjects, each with their own randomly generated prototype, study set, etc. This means that the distance calculated ($d_{ij} = \log(1 + \frac{\sum_m \sqrt{(i_{mx} - j_{mx})^2 + (i_{my} - j_{my})^2}}{9})$) will differ also affecting the averages. This makes sense since an exemplar model uses all 10 individual patterns and comparisons.

The following cell lets you plot the model predictions for the exemplar model fitted to the stimuli that participants in this experiment actually viewed. There is a k and a c parameter for both categorization and recognition.

```
In [32]: c_cat, k_cat, c_rec, k_rec = 2.0, 0.1, 2.0, 0.5
ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement",
                 hue="Condition", data=get_exemplar_results(c_cat, k_cat, c_rec,
ax.set_ylim(0, 1.0);
```



Predictions for the prototype model

The following cells set up the prototype model using the equations described above.

```
In [33]: #####
# prototype model
# stores an average of the study items in memory
# and computes the probability of endorsement
# for each item type
#####
def prototypemodel(filename, c, k):
    data = readfile(filename)
    cond = data[-1][1]
    # average all the old items in memory
    memory = [] # (10, 18) for 10 items of size 18
    for line in data:
        if len(line) == 20 and line[1] == 2:
            memory.append(line[2:])
    prototype = np.array(memory).mean(axis=0) # (18,) average over t
    prototype = prototype.reshape(9, 2)      # (9, 2): 9 dots, aver
    memory = [prototype]

    # prototype items
    proto = []
    for line in data:
        if len(line) == 20 and line[1] == 1:
            item = np.resize(line[2:], (9, 2))
```

```

        pofr = computeresponse(item, memory, c, k)
        proto.append(pofr)
    # print(np.average(proto))

    # old items
    old = []
    for line in data:
        if len(line) == 20 and line[1] == 2:
            item = np.resize(line[2:], (9, 2))
            pofr = computeresponse(item, memory, c, k)
            old.append(pofr)
    # print "p of r", old
    # print(np.average(old))

    # new high items
    newhigh = []
    for line in data:
        if len(line) == 20 and line[1] == 3:
            item = np.resize(line[2:], (9, 2))
            pofr = computeresponse(item, memory, c, k)
            newhigh.append(pofr)
    # print(np.average(newhigh))

    # new low items
    newlow = []
    for line in data:
        if len(line) == 20 and line[1] == 4:
            item = np.resize(line[2:], (9, 2))
            pofr = computeresponse(item, memory, c, k)
            newlow.append(pofr)
    # print(np.average(newlow))

    # random items
    random = []
    for line in data:
        if len(line) == 20 and line[1] == 5:
            item = np.resize(line[2:], (9, 2))
            pofr = computeresponse(item, memory, c, k)
            random.append(pofr)
    # print(np.average(random))

    return [np.average(proto), np.average(newlow), np.average(newhigh), np.a

```

In [34]:

```

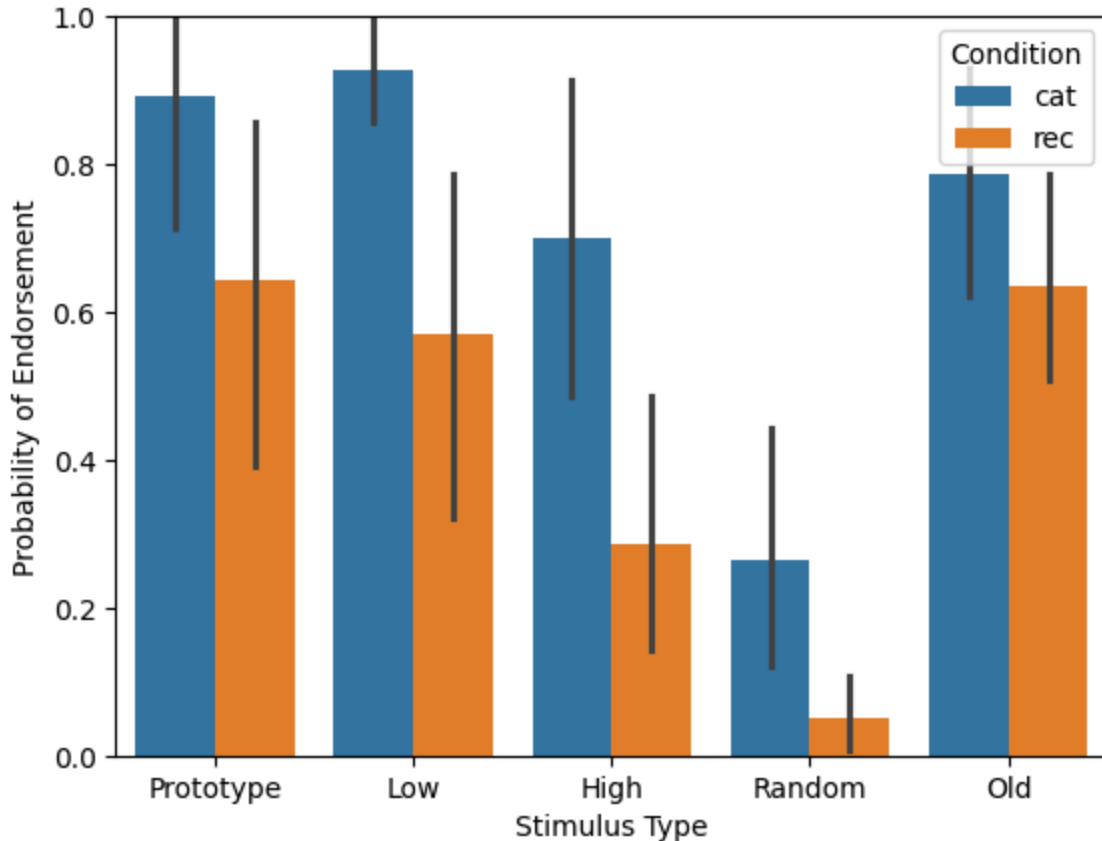
def get_prototype_results(c_cat, k_cat, c_rec, k_rec):
    allres = {fn: readfile(fn) for fn in get_all_filenames('data')}
    cat = []
    rec = []
    for filename in allres.keys():
        if allres[filename][-1][1] == 0:
            res = prototypemodel(filename, c_cat, k_cat)
            cat.append(create_df(filename, 'cat', res[:-2]))
        else:
            res = prototypemodel(filename, c_rec, k_rec)
            rec.append(create_df(filename, 'rec', res[:-2]))

```

```
cat, rec = pd.concat(cat), pd.concat(rec)
return pd.concat([cat, rec])
```

Again lets replot the human results for easy reference.

```
In [35]: ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", hue="Cor
ax.set_ylim(0, 1.0);
```



Problem 3 (10 points)

By hand adjust the setting of the model parameters in the next cell to roughly fit the human data pattern shown above. How close can you get? What parameters did you find (report them) and your assessment of how well they fit. Was it a good fit or are there systematic problems with the fit? In addition, what are the parameter values and do they make sense in light of the equations described above?

Answer

Best fit params found: $c_{cat} = 1.6$, $k_{cat} = 0.03$, $c_{rec} = 1.5$, $k_{rec} = 0.2$

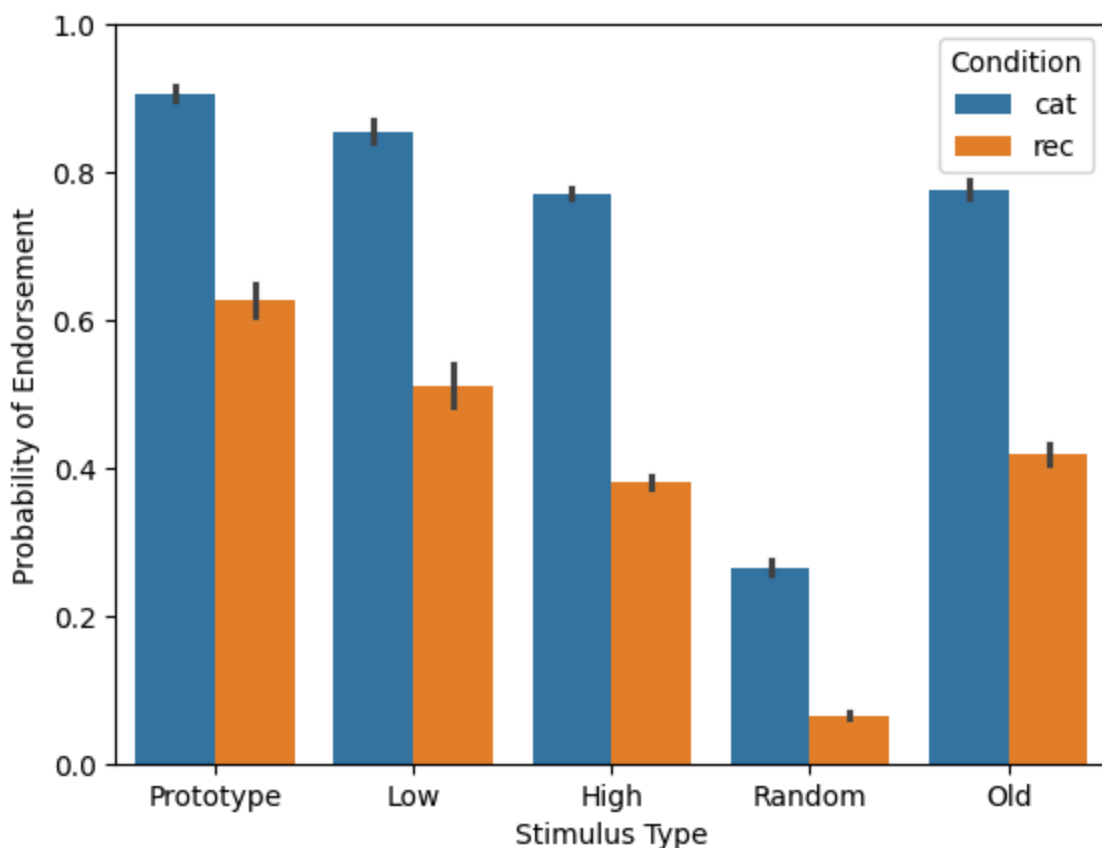
Again I was able to get decently close to that of what was shown for the prototype graph. Categorization fits well with prototype, random, and old showing clear endorsement over their recognition counterparts. However here recognition fits worse as

shown with "old". The reason for this is because the prototype model only stores the averaged pattern, so an old item is just another distortion. It's no closer to the average than a new high distortion. This means that unlike the exemplar model, "old" items don't have an advantage. Like before, keeping $c = 1.6$ (cat) and $c = 1.5$ (rec) allowed the spread to remain like that of the graph given. However unlike the exemplar model, similarity is one term $e^{-c \cdot d_{ip}}$ and not a sum of 10, meaning c must separate near items from far items. In this model k remains much more smaller since a single similarity term is smaller than a sum of ten meaning k has to be small to get high endorsement. Going

back to the equation for prototype:
$$P(A|i) = \frac{e^{-c \cdot d_{ip}}}{e^{-c \cdot d_{ip}} + k}$$

we can see that the criterion (k) is overall much more stricter which makes sense since for a prototype model a single average is used to identify different types, and additionally, "prototype" would have the best fit.

```
In [36]: c_cat, k_cat, c_rec, k_rec = 1.6, 0.03, 1.5, 0.2
res=get_prototype_results(c_cat, k_cat, c_rec, k_rec)
ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", hue="Condition")
ax.set_ylim(0, 1.0);
```



Fitting the models using RMSE

Next we would like to come up with a more quantitative way to assess the quality of the model fits. We need a "goodness of fit" measure, and we'll discuss a few different kinds throughout the semester. One of the most common measures of goodness of fit is the Root Mean Squared Error (RMSE). This measure compares the value of each target y to each prediction $\hat{y}$ (each a vector of N dimensions) using the following equation:

$$RMSE = \sqrt{\frac{\sum_{b=1}^N (y_b - \hat{y}_b)^2}{N}}$$

Often the RMSE is computed between the AVERAGE prediction of the model and the AVERAGE estimates of the behavior to all the subjects in an experiment. Using the code we developed above we can find the average endorsement curves for humans and both models like you see below. Note that this Python code uses the Pandas library. Here is a [short introduction](#) and an explanation of the `.groupby` routine.

```
In [37]: human_results=get_human_results()
exemplar_predictions = get_exemplar_results(c_cat, k_cat, c_rec, k_rec)
prototype_predictions = get_prototype_results(c_cat, k_cat, c_rec, k_rec)
```

```
In [38]: avghuman=human_results.groupby(['Condition', 'Stimulus Type'],as_index=False)
avghuman
```

Out[38]:

	Condition	Stimulus Type	Probability of Endorsement
0	cat	High	0.700000
1	cat	Low	0.928571
2	cat	Old	0.785714
3	cat	Prototype	0.892857
4	cat	Random	0.264286
5	rec	High	0.285714
6	rec	Low	0.571429
7	rec	Old	0.635714
8	rec	Prototype	0.642857
9	rec	Random	0.050000

	Condition	Stimulus Type	Probability of Endorsement
0	cat	High	0.700000
1	cat	Low	0.928571
2	cat	Old	0.785714
3	cat	Prototype	0.892857
4	cat	Random	0.264286
5	rec	High	0.285714
6	rec	Low	0.571429
7	rec	Old	0.635714
8	rec	Prototype	0.642857
9	rec	Random	0.050000

```
In [39]: avgexemplar=exemplar_predictions.groupby(['Condition', 'Stimulus Type'],as_index=False)
avgexemplar
```

Out [39]:

	Condition	Stimulus Type	Probability of Endorsement
0	cat	High	0.956727
1	cat	Low	0.967610
2	cat	Old	0.981388
3	cat	Prototype	0.973529
4	cat	Random	0.773076
5	rec	High	0.812965
6	rec	Low	0.855823
7	rec	Old	0.901524
8	rec	Prototype	0.880373
9	rec	Random	0.400136

```
In [40]: avgprototype=prototype_predictions.groupby(['Condition', 'Stimulus Type'], a
avgprototype
```

Out [40]:

	Condition	Stimulus Type	Probability of Endorsement
0	cat	High	0.771337
1	cat	Low	0.854728
2	cat	Old	0.776662
3	cat	Prototype	0.905904
4	cat	Random	0.264905
5	rec	High	0.380482
6	rec	Low	0.511646
7	rec	Old	0.419295
8	rec	Prototype	0.628160
9	rec	Random	0.065057

Problem 4 (20 points)

First, write a function below called `rmse` that computes the RMSE between two `numpy` vectors.

```
In [41]: def rmse(human, model):
          diff = np.array(human) - np.array(model)
```

```
return np.sqrt(np.sum(diff ** 2) / len(diff))
```

Write your code above. This code will then be used in the provided functions below to evaluate the fit of the prototype and exemplar models. The parameters to the model are provided as a list with `[c_cat, k_cat, c_rec, k_rec]` the implied order.

```
In [42]: def fit_exemplar_model_rmse(params, human_results):
        [c_cat, k_cat, c_rec, k_rec] = params
        predictions = get_exemplar_results(c_cat, k_cat, c_rec, k_rec)
        avgpredict=predictions.groupby(['Condition', 'Stimulus Type'],as_index=False)
        model_results = avgpredict['Probability of Endorsement'].values
        return rmse(human_results, model_results)

def fit_prototype_model_rmse(params, human_results):
    [c_cat, k_cat, c_rec, k_rec] = params
    predictions = get_prototype_results(c_cat, k_cat, c_rec, k_rec)
    avgpredict=predictions.groupby(['Condition', 'Stimulus Type'],as_index=False)
    model_results = avgpredict['Probability of Endorsement'].values
    return rmse(human_results, model_results)
```

```
In [43]: human_results=get_human_results()
        avghuman=human_results.groupby(['Condition', 'Stimulus Type'],as_index=False)
        human_results = avghuman['Probability of Endorsement'].values
        print(fit_exemplar_model_rmse([2.0, 2.0, 2.0, 2.0], human_results))
        print(fit_prototype_model_rmse([2.0, 2.0, 2.0, 2.0], human_results))
```

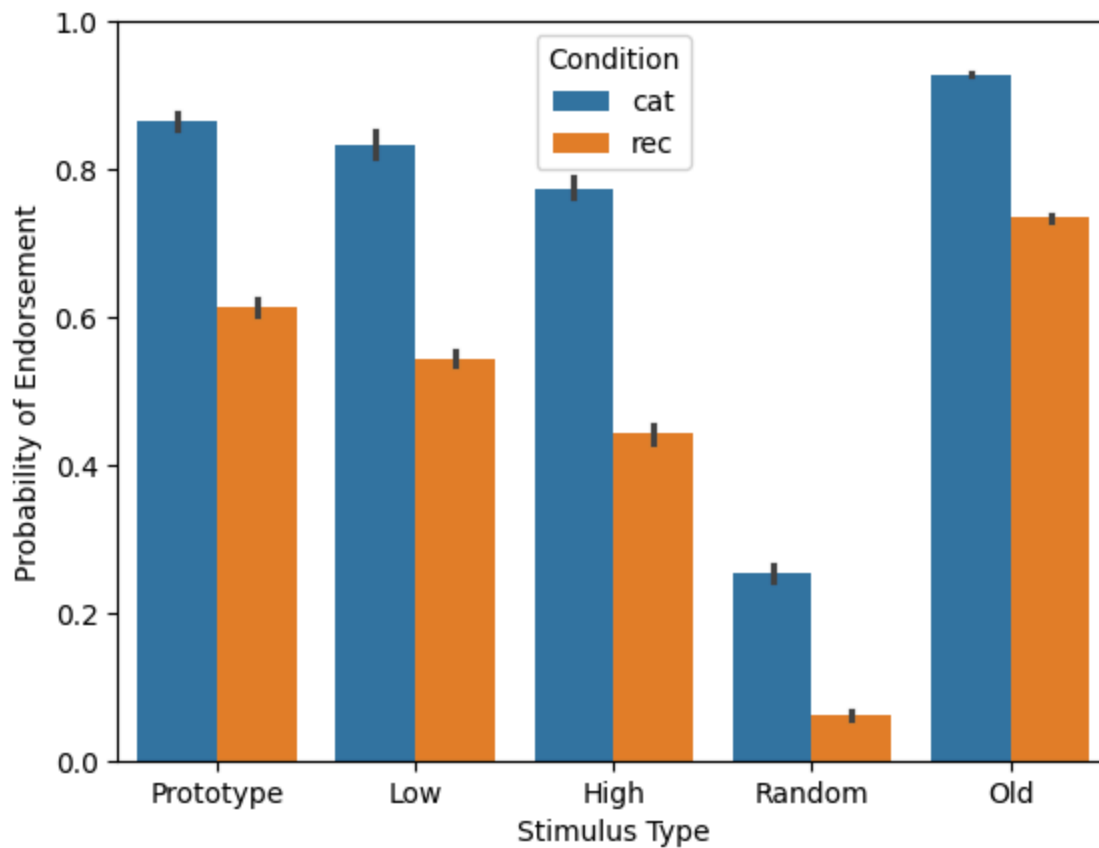
0.41994732408249763

0.586486858992767

Next adjust the parameters by hand for both the exemplar and prototype models to find values that appear to minimize the RMSE. Copy the code above for plotting the predictions of the models given your best fit parameters. Which model do you think fits better according to this fit statistic?

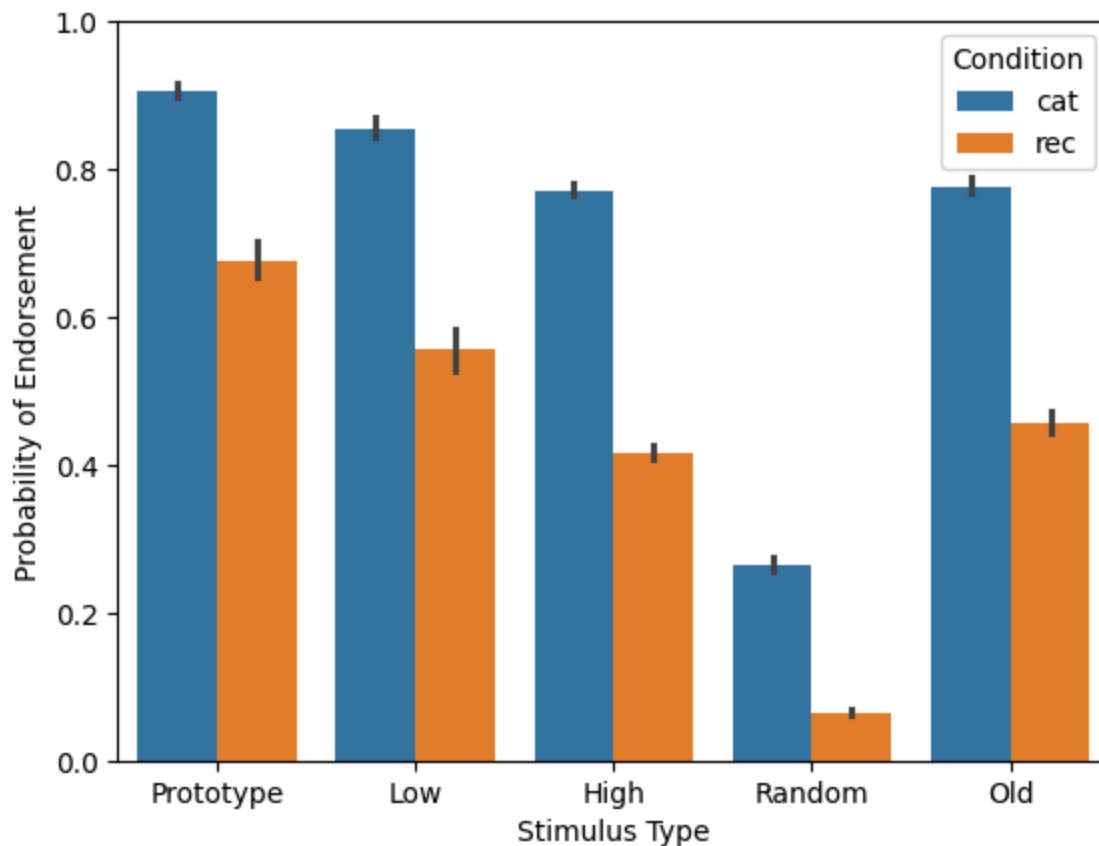
```
In [44]: # Exemplar model with best fit parameters
        c_cat, k_cat, c_rec, k_rec = 2.0, 0.1, 2.0, 0.5
        ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", hue="Condition",
                        data=get_exemplar_results(c_cat, k_cat, c_rec, k_rec))
        ax.set_ylim(0, 1.0);
        print('Exemplar RMSE:', fit_exemplar_model_rmse([c_cat, k_cat, c_rec, k_rec]
```

Exemplar RMSE: 0.08477639500607148



```
In [45]: # Prototype model with best fit parameters
c_cat, k_cat, c_rec, k_rec = 1.6, 0.03, 1.6, 0.15
ax = sns.barplot(x="Stimulus Type", y="Probability of Endorsement", hue="Condition",
                 data=get_prototype_results(c_cat, k_cat, c_rec, k_rec))
ax.set_ylim(0, 1.0);
print('Prototype RMSE:', fit_prototype_model_rmse([c_cat, k_cat, c_rec, k_rec]))
```

Prototype RMSE: 0.07805038838419211



Answer

Best fit params found

Model	c_cat	k_cat	c_rec	k_rec	RMSE
Exemplar	2.0	0.1	2.0	0.5	0.0848
Prototype	1.6	0.03	1.6	0.15	0.0781

By RMSE the prototype model fits better (0.0781 vs. 0.0848), but only slightly, so I would not say either model clearly wins. I found these values by adjusting k until the overall bar height and the random bar matched the human data, then adjusting c until the gap between the high bars and low bars matched. The models miss in different ways. The exemplar model over-predicts old items and high distortions, because an old item has an exact copy in memory ($d = 0$) and so gets a large boost in summed similarity. The prototype model fits categorization well, including the prototype advantage, but does worse for recognition since it stores only an averaged pattern, so old items look no more familiar than new high distortions.

Problem 5 (10 points)

Read about the scipy `fmin` function. Use `fmin` to algorithmically search for the best

parameters for each model using the RMSE evaluation function described above. Make sure that you save the optimized values to the variables `rmse\_exemplar` and `rmse\_prototype`

To unpack the outputs properly, write as:

```
param_opt_exemplar, rmse_exemplar, *_ = fmin(...your code here...,
full_output=True)
param_opt_prototype, rmse_prototype, *_ = fmin(...your code here...,
full_output=True)
```

Remidner

- **Do NOT clear the output when submitting.** Submit the notebook with the output of each cell. The grader may use these values.

```
In [46]: ### YOUR ANSWER HERE
param_opt_exemplar, rmse_exemplar, *_ = fmin(fit_exemplar_model_rmse, [2.0,
param_opt_prototype, rmse_prototype, *_ = fmin(fit_prototype_model_rmse, [1.
### END ANSWER
print('The RMSE for the exemplar model is: ', rmse_exemplar)
print('The RMSE for the prototype model is: ', rmse_prototype)
```

Optimization terminated successfully.

Current function value: 0.076819

Iterations: 106

Function evaluations: 181

Optimization terminated successfully.

Current function value: 0.077796

Iterations: 89

Function evaluations: 155

The RMSE for the exemplar model is: 0.07681866357417835

The RMSE for the prototype model is: 0.07779553420996524